

EECS 836 Homework 2

JOHNPAUL MBAGWU

1

Problem 1: Neural Networks

Solution (1): Why and how do we include a bias term in the input and in the hidden-layer?

Why? Bias terms allow the network to shift the activation function independently of the input, making the model more flexible and able to learn patterns beyond the origin. **How?** The bias is included as an extra node with a constant value of 1 in both the input and hidden layers.

Solution (2): Why do we need to use nonlinear activation functions in our neural net?

Without nonlinearity, multiple layers collapse into a single linear transformation. Nonlinear activation functions enable the model to learn complex representations.

Solution (3): Compute v_1, v_2, v_3 for given inputs $x_1 = 1, x_2 = 2$

Using the weight matrix:

$$\alpha = \begin{bmatrix} 0 & 1 & 2 \\ 2 & 1 & 0 \\ 0 & 2 & 0 \end{bmatrix}$$

And input:

$$x = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}$$

Each v_j is computed as:

$$v_j = \sum_{i=0}^2 \alpha_{j,i} x_i$$

$$v_1 = 0(1) + 1(1) + 2(2) = 5$$

$$v_2 = 2(1) + 1(1) + 0(2) = 3$$

$$v_3 = 0(1) + 2(1) + 0(2) = 2$$

Problem 2: CNN

Solution (1)

Dimensions of the output = 3x3

As a result, the generated image's lower right corner is -22.

$$\begin{bmatrix} -9 & 0 \\ 3 & 7 \end{bmatrix} * \begin{bmatrix} 1 & 2 \\ -2 & -1 \end{bmatrix} = (-9 \times 1) + (0 \times 2) + (3 \times -2) + (7 \times -1) = -22$$

Solution (2)

While the filter size remains unchanged, the resulting picture dimensions are 6x6 when padding = 1.

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 3 & -2 & 4 & 0 \\ 0 & 0 & 8 & 6 & 5 & 0 \\ 0 & 2 & 1 & -9 & 0 & 0 \\ 0 & 4 & -1 & 3 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ -2 & -1 \end{bmatrix}$$

The dimensions of the output = 5x5

7 is the new value in the bottom right corner.

$$\begin{bmatrix} 7 & 0 \\ 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 2 \\ -2 & -1 \end{bmatrix} = (7 \times 1) + (0 \times 2) + (0 \times -2) + (0 \times -1) = 7$$

Solution (3)

Suppose we have below 2-channel input I and filter F as follows. What is the dimensions of the output, assume padding=0 and stride=1, and what is the value in the bottom right corner of the output image?

$$I[0, :, :] = \begin{bmatrix} 4 & 2 & 1 & 3 \\ 6 & 1 & -2 & -3 \\ -1 & 3 & 4 & 0 \end{bmatrix}$$

$$F[0, :, :] = \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix}$$

$$I[1, :, :] = \begin{bmatrix} -2 & 1 & 4 & 5 \\ 0 & 6 & 3 & 6 \\ 4 & -1 & 1 & 1 \end{bmatrix}$$

$$F[1, :, :] = \begin{bmatrix} 1 & 0 \\ 1 & -1 \end{bmatrix}$$

The dimensions of the output = 2 x 3

For the input zero $I[0, :, :]$ and filter zero $F[0, :, :]$, we have:

$$\begin{bmatrix} 4 & 2 & 1 \\ 6 & 1 & -2 \\ -1 & 3 & 4 \end{bmatrix} * \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & -2 & -8 \\ 7 & 9 & 4 \end{bmatrix}$$

Bottom right corner:

$$\begin{bmatrix} -2 & -3 \\ 0 & 4 \end{bmatrix} * \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} = (-2 \times 1) + (-3 \times -2) + (4 \times 0) + (0 \times 1) = -2 + 6 + 0 + 0 = 4$$

For the input one $I[1, :, :]$ and filter one $F[1, :, :]$:

$$\begin{bmatrix} -2 & 1 & 4 \\ 0 & 6 & 3 \\ 4 & -1 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} -8 & 4 & 1 \\ 5 & 4 & 3 \end{bmatrix}$$

Bottom right corner:

$$\begin{bmatrix} 3 & 6 \\ 1 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ 1 & -1 \end{bmatrix} = (3 \times 1) + (6 \times 0) + (1 \times 1) + (1 \times -1) \\ = 3 + 0 + 1 - 1 = 3$$

The two channels' output is:

$$\begin{bmatrix} 1 & -2 & -8 \\ 7 & 9 & 4 \end{bmatrix} + \begin{bmatrix} -8 & 4 & 1 \\ 5 & 4 & 3 \end{bmatrix} = \begin{bmatrix} -7 & 2 & -7 \\ 12 & 13 & 7 \end{bmatrix}$$

Hence, the bottom right corner of the output image = 4 + 3 = 7.

Problem 3: Recurrent Neural Network

Solution (1)

$$p = u_1 + 0 = u_1$$

$$q = w \cdot p = w \cdot u_1$$

$$r = u_2 + q = u_2 + w \cdot u_1$$

$$s = w \cdot r = w \cdot (u_2 + w \cdot u_1) = w \cdot u_2 + w^2 \cdot u_1$$

$$t = u_3 + s = u_3 + w \cdot u_2 + w^2 \cdot u_1$$

$$y = w \cdot t = w \cdot (u_3 + w \cdot u_2 + w^2 \cdot u_1) = w \cdot u_3 + w^2 \cdot u_2 + w^3 \cdot u_1$$

Solution (2)

Recurrent Neural Networks (RNNs) have memory because they maintain a hidden state that preserves information from previous inputs through recurrent connections. This hidden state acts as a memory, enabling RNNs to process sequential data while integrating contextual information from earlier time steps. However, a key limitation of traditional RNNs is the vanishing gradient problem, where gradients shrink exponentially during backpropagation, making it difficult for the network to learn long-term dependencies. In contrast, Long Short-Term Memory (LSTM) networks overcome this challenge by using specialized gating mechanisms to regulate information flow, allowing them to capture and retain long-range dependencies more effectively than standard RNNs.

Solution (3) : Given the LSTM architecture as shown in Figure 3. Suppose you want the memory cell to sum its inputs over time. What values should the input gate and forget gate take?

To enable the memory cell to sum inputs over time:

The input gate must be fully open to allow all information from the current time step to be written to the memory cell. Therefore, the input gate should be set to 1. The forget gate must also be fully open to retain all information from previous time steps. Hence, the forget gate should be set to 1. In conclusion, for the memory cell to accumulate inputs over time, both the input gate and forget gate should be set to 1.

Problem 4: Graph Neural Network

Solution (1) : What is the adjacency matrix of the graph?

The adjacency matrix is a fundamental data structure used to represent a graph in computer memory. For the given graph, the adjacency matrix has dimensions 4×4 , corresponding to the four nodes present in the graph. Each row and column of the matrix correspond to a node, where an entry of 1 indicates the presence of an edge between two nodes, and an entry of 0 signifies the absence of an edge. Using this convention, the adjacency matrix for the graph is constructed as follows:

	A	B	C	D
A	0	1	0	0
B	1	0	1	0
C	0	1	0	1
D	0	0	1	0

Solution (2) :

To calculate the first GCN layer output h_v^1 for nodes A, B, C, D , I apply the GCN layer equation with W_1 and $\sigma = ReLU$:

For Node A

$$h_A^1 = ReLU \left(W_1 * \left(\frac{v_A}{\sqrt{N(A)}||N(v_A)}} + \frac{v_B}{\sqrt{N(B)}||N(v_A)}} \right) \right)$$

$$h_A^1 = ReLU \left(\begin{bmatrix} 3 & 3 \\ 2 & 1 \end{bmatrix} * \left(\frac{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}{\sqrt{(1 * 1)}} + \frac{\begin{bmatrix} -1 \\ 2 \end{bmatrix}}{\sqrt{(2 * 1)}} \right) \right)$$

$$h_A^1 = ReLU \left(\begin{bmatrix} 3 & 3 \\ 2 & 1 \end{bmatrix} * \left(\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} -1 \\ 2 \end{bmatrix} / \sqrt{2} \right) \right)$$

$$h_A^1 = ReLU \left(\begin{bmatrix} 3 & 3 \\ 2 & 1 \end{bmatrix} * \left(\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} -1 \\ 2 \end{bmatrix} / \sqrt{2} \right) \right)$$

$$h_A^1 = ReLU \left(\left(\begin{bmatrix} 9 + 3/\sqrt{2} \\ 9 + 6/\sqrt{2} \end{bmatrix} \right) / 4 \right)$$

$$h_A^1 = ReLU \left(\begin{bmatrix} 11.121 \\ 4 \end{bmatrix} \right)$$

$$h_A^1 = \begin{bmatrix} 11.121 \\ 4 \end{bmatrix}$$

For Node B

$$h_B^1 = ReLU \left(W_1 * \left(\frac{v_A}{\sqrt{N(A)} || N(v_B)} + \frac{v_B}{\sqrt{N(B)} || N(v_B)} + \frac{v_C}{\sqrt{N(C)} || N(v_B)} \right) \right)$$

$$h_B^1 = ReLU \left(\begin{bmatrix} 3 & 3 \\ 2 & 1 \end{bmatrix} * \left(\frac{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}{\sqrt{1 \times 2}} + \frac{\begin{bmatrix} -1 \\ 2 \end{bmatrix}}{\sqrt{2 \times 2}} + \frac{\begin{bmatrix} 2 \\ 2 \end{bmatrix}}{\sqrt{2 \times 2}} \right) \right)$$

$$h_B^1 = ReLU \left(\begin{bmatrix} 13.864 \\ 5.829 \end{bmatrix} \right)$$

$$h_B^1 = \begin{bmatrix} 13.864 \\ 5.829 \end{bmatrix}$$

For Node C

$$h_C^1 = ReLU \left(W_1 * \left(\frac{v_B}{\sqrt{N(B)} || N(v_C)} + \frac{v_C}{\sqrt{N(C)} || N(v_C)} + \frac{v_D}{\sqrt{N(D)} || N(v_C)} \right) \right)$$

$$h_C^1 = ReLU \left(\begin{bmatrix} 3 & 3 \\ 2 & 1 \end{bmatrix} * \left(\frac{\begin{bmatrix} -1 \\ 2 \end{bmatrix}}{\sqrt{2 \times 2}} + \frac{\begin{bmatrix} 2 \\ 2 \end{bmatrix}}{\sqrt{2 \times 2}} + \frac{\begin{bmatrix} -3 \\ 1 \end{bmatrix}}{\sqrt{1 \times 2}} \right) \right)$$

$$h_C^1 = ReLU \left(\begin{bmatrix} 5.379 \\ 0.172 \end{bmatrix} \right)$$

$$h_C^1 = \begin{bmatrix} 5.379 \\ 0.172 \end{bmatrix}$$

For Node D

$$h_D^1 = ReLU \left(W_1 * \left(\frac{v_C}{\sqrt{N(C)} \|N(v_D)\|} + \frac{v_D}{\sqrt{N(D)} \|N(v_D)\|} \right) \right)$$

$$h_D^1 = ReLU \left(\begin{bmatrix} 3 & 3 \\ 2 & 1 \end{bmatrix} * \left(\frac{\begin{bmatrix} 2 \\ 2 \end{bmatrix}}{\sqrt{2 \times 1}} + \frac{\begin{bmatrix} -3 \\ 1 \end{bmatrix}}{\sqrt{1 \times 1}} \right) \right)$$

$$h_B^1 = ReLU \left(\begin{bmatrix} 3 & 3 \\ \frac{2}{2} & 1 \end{bmatrix} * \begin{bmatrix} \frac{2}{\sqrt{2}} \\ \frac{-3}{2} \end{bmatrix} \right)$$

$$h_B^1 = ReLU \left(\begin{bmatrix} \frac{3}{2} & 3 \\ 1 & 1 \end{bmatrix} * \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} -3 \\ 2 \end{bmatrix} \right) \right)$$

$$h_B^1 = ReLU \left(\begin{bmatrix} \frac{3}{2} & 3 \\ 1 & 1 \end{bmatrix} * \left(\frac{\sqrt{2}}{2} \begin{bmatrix} 3 & 3 \\ \frac{2}{2} & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 & 3 \\ \frac{2}{2} & 1 \end{bmatrix} \begin{bmatrix} -3 \\ 1 \end{bmatrix} \right) \right)$$

$$h_B^1 = ReLU \left(\frac{\sqrt{2}}{2} \begin{bmatrix} 6+6 \\ 4+2 \end{bmatrix} + \begin{bmatrix} -9+6 \\ -6+2 \end{bmatrix} \right)$$

$$h_B^1 = ReLU \left(\frac{\sqrt{2}}{2} \begin{bmatrix} 12 \\ 6 \end{bmatrix} + \begin{bmatrix} -3 \\ -4 \end{bmatrix} \right)$$

$$h_B^1 = ReLU \left(\begin{bmatrix} 8.485 \\ 4.243 \end{bmatrix} + \begin{bmatrix} -3 \\ -4 \end{bmatrix} \right)$$

$$h_B^1 = ReLU \left(\begin{bmatrix} 5.485 \\ 0.243 \end{bmatrix} \right)$$

$$h_B^1 = \begin{bmatrix} 5.485 \\ 0.243 \end{bmatrix}$$

Solution (3) :

For node A:

$$h_A^2 = ReLU \left(W_2 * \left(\frac{h_A^1}{\sqrt{N(A)}||N(v_A)}} + \frac{h_B^1}{\sqrt{N(B)}||N(v_A)}} \right) \right)$$

$$h_A^2 = ReLU \left(\begin{bmatrix} 2 & 5 \\ 4 & 2 \end{bmatrix} * \begin{bmatrix} \frac{11.121}{4} \\ \frac{13.864}{\sqrt{2}} \end{bmatrix} \right)$$

$$h_A^2 = ReLU \left(\begin{bmatrix} 42.242 \\ 52.484 \end{bmatrix} + \begin{bmatrix} 40.215 \\ 47.457 \end{bmatrix} \right)$$

$$h_A^2 = ReLU \left(\begin{bmatrix} 82.457 \\ 99.941 \end{bmatrix} \right)$$

$$h_A^2 = \begin{bmatrix} 82.457 \\ 99.941 \end{bmatrix}$$

For node B:

$$h_B^2 = ReLU \left(W_2 * \left(\frac{h_A^1}{\sqrt{N(A)}||N(v_B)}} + \frac{h_B^1}{\sqrt{N(B)}||N(v_B)}} + \frac{h_C^1}{\sqrt{N(C)}||N(v_B)}} \right) \right)$$

$$h_B^2 = ReLU \left(\begin{bmatrix} 2 & 5 \\ 4 & 2 \end{bmatrix} \left(\frac{11.121}{4} + \frac{13.864}{\sqrt{2}} + \frac{5.379}{4} \right) \right)$$

$$h_B^2 = ReLU \left(\begin{bmatrix} 42.242 \\ 52.484 \end{bmatrix} + \begin{bmatrix} 56.873 \\ 21.86 \end{bmatrix} + \begin{bmatrix} 5.809 \\ 10.93 \end{bmatrix} \right)$$

$$h_B^2 = ReLU \left(\begin{bmatrix} 64.115 \\ 81.599 \end{bmatrix} \right)$$

$$h_B^2 = \begin{bmatrix} 64.115 \\ 81.599 \end{bmatrix}$$

Solution (4) : From the perspective of “message passing”, what does the k-th layer of GCN achieve in graph representation learning?

In the k -th layer of a Graph Convolutional Network (GCN), each node updates its representation by aggregating messages from its neighboring nodes as well as retaining its own representation from the $(k-1)$ -th layer. This aggregation process is guided by a learnable weight matrix W_k and can be normalized using node degrees.

After aggregation, the updated representation undergoes a non-linear activation function, specifically the ReLU function in this case, to produce the node's refined representation in the k -th layer. This iterative message-passing mechanism enables nodes to progressively integrate and propagate information from their local neighborhoods, allowing the model to effectively learn both node features and the underlying graph structure. As the network depth increases, nodes incorporate information from more distant regions in the graph. This facilitates the learning of higher-order representations, capturing more global structural patterns and enhancing the model's ability to encode complex graph relationships.